

Ship Kanban to EC2

Getting the Next.js app and its MongoDB container running on a fresh Ubuntu EC2 instance, built on the server from source.

Host Ubuntu / EC2 Image node:22-alpine Port 3000 Containers app + mongo

01 Open the ports

AWS Console – no terminal needed

EC2 → your instance → **Security** tab → click the security group → **Edit inbound rules** → Add rule:

TYPE	PORT	SOURCE
SSH	22	My IP (already there)
Custom TCP	3000	0.0.0.0/0

Do not open 27017. MongoDB stays private, reachable only by the app container over Docker's internal network.

02 Install Docker

```
$ sudo apt update && sudo apt upgrade -y
$ curl -fsSL https://get.docker.com | sudo sh
$ sudo usermod -aG docker ubuntu
```

Log out and SSH back in before continuing — the group change only applies to a new session. Then confirm it works:

```
$ docker run hello-world
```

03

Add swap

Required on t2.micro / t3.micro (1 GB RAM)

One gigabyte is not enough for `next build` — the build gets killed with no useful error. Two gigabytes of swap fixes it.

```
$ sudo fallocate -l 2G /swapfile
$ sudo chmod 600 /swapfile
$ sudo mkswap /swapfile
$ sudo swapon /swapfile
$ echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

The last line makes the swap survive a reboot. Skip this whole step only if the instance has 2 GB of RAM or more.

04

Clone the code

```
$ git clone https://github.com/Nasir1504/<repo>.git
$ cd <repo>
```

For a private repo, Git asks for a username and password — the password is a GitHub **personal access token**, not your account password.

05

Write the compose file

Create it with `nano docker-compose.yml`, paste, then save with `Ctrl+O`, `Enter`, `Ctrl+X`.

```
docker-compose.yml

services:
  mongo:
    image: mongo:8
    restart: always
    volumes:
      - mongo-data:/data/db

  app:
    build: .
    restart: always
    ports:
      - "3000:3000"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/kanban
    depends_on:
      - mongo

volumes:
  mongo-data:
```

- Inside Docker, `mongo` is the hostname — the service name. Not `localhost`.
- `mongo-data` is a named volume, so your tasks survive restarts and `docker compose down`.
- The mongo service has no `ports:` block on purpose — that is what keeps it off the internet.

Commit this file to the repo afterwards so you never hand-write it on a server again.

06

Build and run

```
$ docker compose up -d --build
$ docker compose ps          # both should say running
```

The first build takes roughly three to five minutes.

07

Open it

```
http://<ec2-public-ip>:3000
```

Use **http**, not **https** — there is no certificate yet. The address is the “Public IPv4 address” on the instance page.

When something breaks

```
$ docker compose logs -f app      # app errors
$ docker compose logs -f mongo    # database errors
```

SYMPTOM	CAUSE
Build “Killed”, no error	Out of memory — go back to step 03
Page never loads	Port 3000 missing from the security group
App starts, no data	<code>MONGODB_URI</code> host is wrong — must be <code>mongo</code>
<code>permission denied</code> on docker	You did not re-open the SSH session after step 02

Redeploy after a code change

```
$ git pull && docker compose up -d --build
```

Two things to watch in this repo

- **nasir-ec2-server.pem** sits in the project folder. It is ignored by both `.gitignore` and `.dockerignore`, so nothing is leaking — just never force-add it.
- **Using MongoDB Atlas instead?** Drop the `mongo` service entirely and put the Atlas connection string in `MONGODB_URI` — then add the EC2 public IP to Atlas Network Access, or connections will hang with no error.